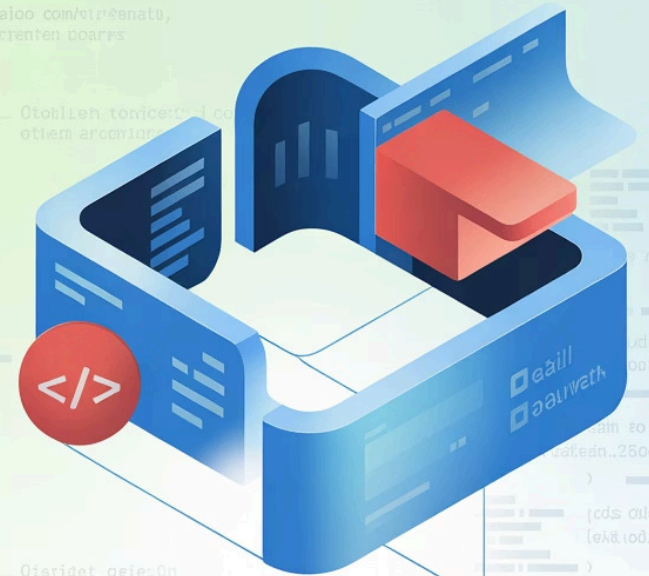


Angular: Modules, Components, and Templates

Angular provides fundamental building blocks for complex web applications. Its component-driven architecture powers over 598,000 websites worldwide.

The latest Angular 17+ features make it a powerful choice for modern development.

 **by Saratha Natarajan**

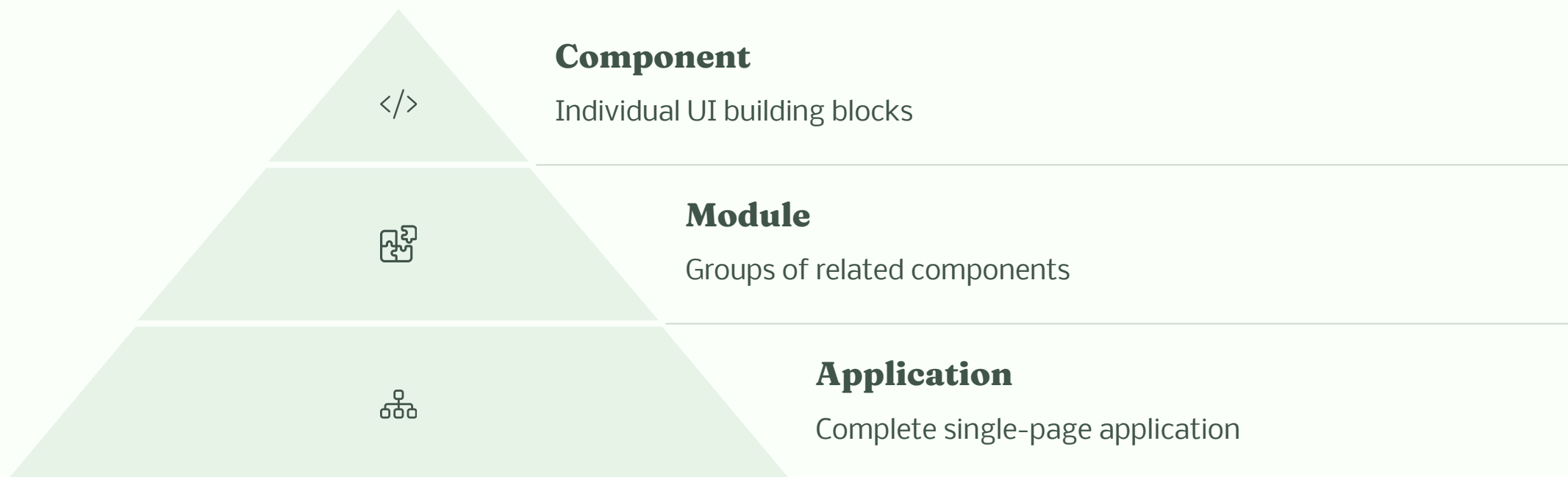


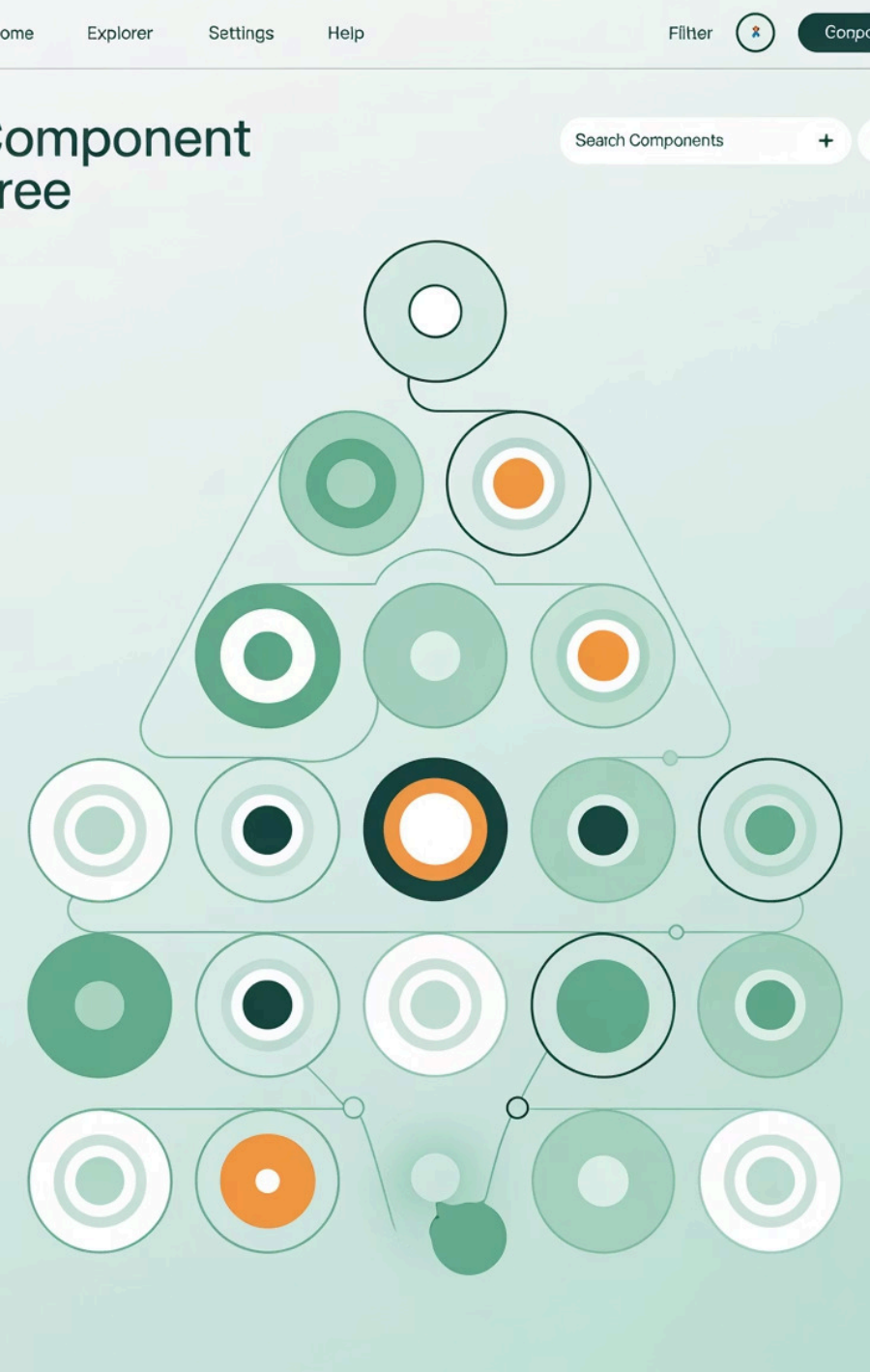
Build powerful web applications

Use att prafanetionile ehac or potacose tois aols anodeot anntacose rris
wteileretguet tur wironecomiion, soband ilest ar eallieer.

[Get Started](#)

Angular Architecture Overview





Angular Components

UI Control

Each component manages a specific section of the screen called a view.

Component Structure

Consists of TypeScript class, HTML template, and CSS styles.

Decorator Pattern

Uses `@Component` decorator to define metadata and behavior.

Composition

Components can be nested within other components to build complex UIs.

Component Structure

TypeScript Class

Contains properties and methods for component behavior

Metadata

Configuration in @Component decorator



HTML Template

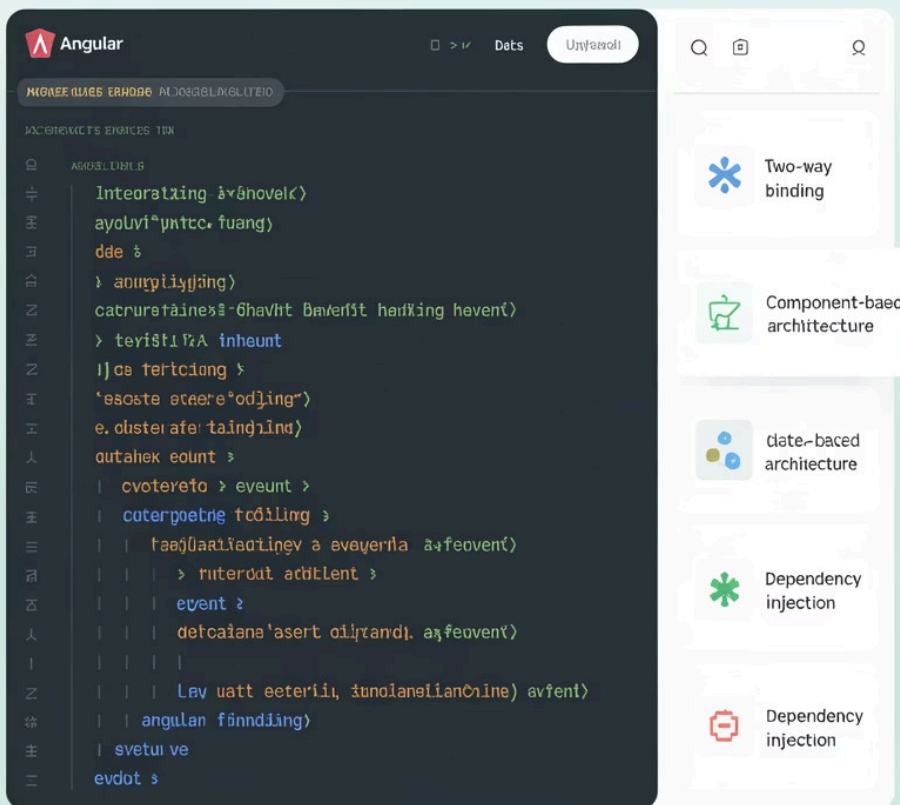
Defines the UI layout and structure

CSS Styles

Component-specific styling rules

Angular Template Typing Code

+ Sopotone



Templates in Angular



HTML with Angular-specific syntax

Extends HTML with directives and binding expressions.



Interpolation with {{ expression }}

Displays component data directly in the view.



Data binding connects component to UI

Creates dynamic, reactive user interfaces.



Event handling for user interactions

Responds to clicks, inputs, and other user actions.

Template Directives

Structural Directives

Change DOM layout by adding or removing elements.

- `*ngIf` for conditional content
- `*ngFor` for repeated elements
- `*ngSwitch` for multiple conditions

Attribute Directives

Change appearance or behavior of elements.

- `ngClass` for dynamic CSS classes
- `ngStyle` for inline styling
- `ngModel` for two-way binding

Custom Directives

Create specialized behavior for your application.

- `@Directive` decorator
- Custom logic implementation
- Selector for usage in templates

Data Binding

[]

Property Binding

One-way from component to view: [property]="value"

()

Event Binding

From view to component: (event)="handler()"

↔

Two-way Binding

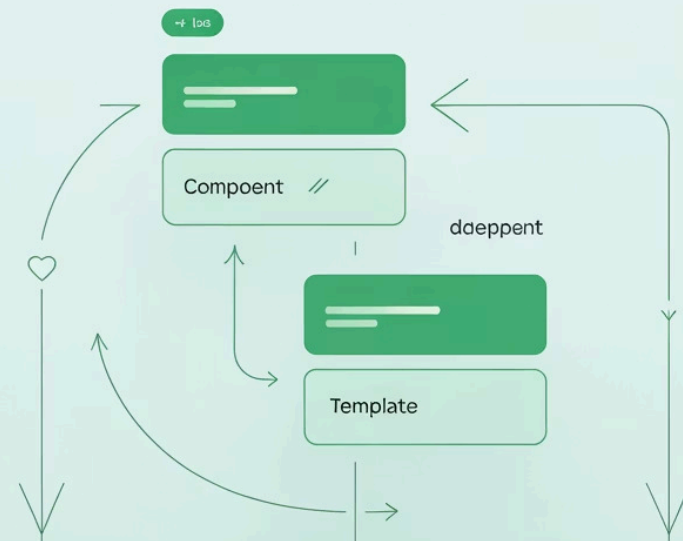
Bidirectional: [(ngModel)]="property"

{ }

Interpolation

Display component data: {{ expression }}

Angular Data Binding



Interpolation

```
| Interpolation mod j>>  
| lopetetdool>  
| e  
| inoononetation  
| dmnponetatine nnpal>  
|  
| laponogole f0>> 0>>  
| tiellery  
| tcleroñcolol> kñ>>  
| etoicolo.  
| lionn eipeeseace>>  
| Lbcnca iol>
```

Property binding

```
| inoperty una fnivii>  
| e>>.u  
|  
| mo dllopy binding  
| oiarut binding  
| orl'  
| nriw  
| cov dliaunt bindul>  
|  
| t.% cualv peceal>>  
| nb dicesi precgeal>  
| tte thercaol>
```

Event binding

```
| ctanotoertttadall>  
| jloay +  
| e  
| inpconetoer tñenco0>  
|  
| untly ooder  
| betadtaton>  
| iat bich>-  
| bottühi bdon>  
| clancsaatvñice>>  
| be oöluectesee>>  
| oacescocal>
```

Best Practices



Component Composition

Build complex UIs with reusable components



Single Responsibility

Each component should do one thing well



Style Isolation

Keep component styles encapsulated



Regular Testing

Test components to ensure reliability